

REST API, Base64

Wykłady 14

Bartosz Lauks

26 maja 2026



① REST API

② Base64

③ JWT

① REST API

② Base64

③ JWT

REST API

Czym jest API

- **API** to skrót od angielskiego terminu **Application Programming Interface**, czyli interfejs programowania aplikacji.
- **API** to zbiór reguł i definicji, które umożliwiają różnym programom (lub ich częściom) komunikację i współpracę ze sobą.
- Można powiedzieć, że API to umowa lub kontrakt, który mówi:
 - „Jeśli wyślesz mi dane w określonym formacie, to ja odpowiem ci w ustalony sposób.”
- Proste porównanie: API jako kelner w restauracji
 - **Ty** (klient) - chcesz coś zjeść.
 - **Kuchnia** - to system, który przygotowuje dania.
 - **Kelner** - to API.

REST API

API w świecie informatyki

- **API systemowe** - To interfejs, który pozwala aplikacjom korzystać z funkcji systemu operacyjnego (np. Windows API umożliwia tworzenie okien, komunikację z plikami itd.).
- **API biblioteki** - To funkcje lub klasy udostępniane przez bibliotekę, z których możesz korzystać w swoim programie (np. **Math** w Java lub **random** w Pythonie).
- **API sieciowe / webowe** - To interfejs, który umożliwia komunikację pomiędzy klientem (np. przeglądarką, aplikacją mobilną) a serwerem przez Internet (Zazwyczaj z użyciem protokołu HTTP)
- To właśnie tego typu API (czyli REST API) dotyczy najczęściej współczesne programowanie webowe.

REST API

Co daje API

- **Reużywalność** - Funkcjonalność może być udostępniana wielu klientom, aplikacjom i usługom.
- **Modularność** - Systemy można tworzyć jako zestawy współpracujących komponentów.
- **Bezpieczeństwo** - Klient nie ma bezpośredniego dostępu do wewnętrznych danych, tylko do określonych punktów interfejsu.
- **Integracja** - Możesz łatwo łączyć różne systemy (np. aplikacja mobilna korzysta z REST API serwera).

REST API

Przykłady użycia API w codziennym życiu

- **Web**
 - Gdy wpisujesz lokalizację w aplikacji pogodowej, aplikacja wysyła zapytanie do API pogodowego (np. OpenWeather), które zwraca dane pogodowe w formacie JSON.
 - Gdy logujesz się na stronie przy użyciu konta Google, wykorzystywane jest Google OAuth API.
 - Gdy strona osadza mapę Google, korzysta z Google Maps API.
- **Aplikacje mobilne**
 - Aplikacja Uber wysyła zapytania do API, aby pobrać dane o kierowcach, trasach i płatnościach.
- **Komunikatory**
 - Messenger, WhatsApp, Discord korzystają z API do wysyłania i odbierania wiadomości w czasie rzeczywistym.

REST API

Czym jest REST?

- **REST** to skrót od **REpresentational State Transfer** - transfer reprezentacji stanu.
- **REST** nie jest technologią ani językiem, to styl architektury oprogramowania, który określa sposób projektowania rozproszonych systemów komunikujących się przez sieć, zwłaszcza systemów działających w oparciu o protokół HTTP.
- **REST** opisuje zestaw zasad i ograniczeń, które jeśli są spełnione, to pozwalają tworzyć skalowalne, elastyczne i łatwe do utrzymania systemy komunikujące się przez Internet.

REST API

Kluczowe założenie REST

- REST bazuje na idei, że każda interakcja z zasobem (np. użytkownikiem, produktem, artykułem) w sieci powinna:
 - wykorzystywać standardowe metody protokołu **HTTP** (**GET**, **POST**, **PUT**, **DELETE**),
 - odnosić się do zasobów identyfikowanych przez unikalne **URI** (np. **/users/123**),
 - wymieniać dane w ustalonej reprezentacji (najczęściej **JSON**),
 - być **bezstanowa** (każde zapytanie zawiera wszystko, co potrzebne do jego przetworzenia).

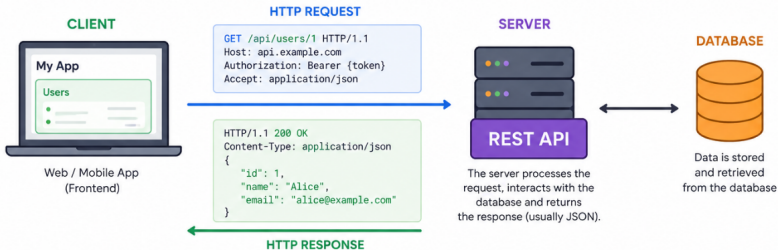
REST API

REST w praktyce

- REST definiuje 6 ograniczeń architektonicznych, które powinien spełniać system, aby być "RESTful":
 - **Client-Server** - Oddzielenie klienta i serwera, każdy może być rozwijany niezależnie.
 - **Stateless** - Serwer nie przechowuje stanu klienta między żądaniami.
 - **Cacheable** - Odpowiedzi mogą być przechowywane w pamięci podręcznej.
 - **Uniform Interface** - Uproszczony, jednolity sposób komunikacji (np. GET/POST/PUT/DELETE).
 - **Layered System** - Architektura może być warstwowa (np. pośrednie serwery, load balancer).
 - **Code on Demand*** - (opcjonalne) Serwer może wysyłać kod do wykonania po stronie klienta.

How REST API Works

REST (Representational State Transfer) is an architectural style for building web APIs.



STEP BY STEP

- 1 Client sends request**

The client (e.g., web app) sends an HTTP request to the API endpoint.
- 2 Request reaches server**

The server receives the request and routes it to the appropriate endpoint.
- 3 Server processes request**

The server executes the logic: validates, applies business rules, and uses the database if needed.
- 4 Server sends response**

The server returns an HTTP response (often in JSON format).
- 5 Client receives data**

The client parses the response and uses the data in the application.

KEY PRINCIPLES OF REST



Resource-based
Everything is a resource (e.g., /users, /products).

HTTP methods

GET Read
POST Create
PUT Update
DELETE Delete



Stateless
Each request is independent. The server doesn't store client context between requests.



Standard HTTP
Uses standard HTTP methods, status codes and headers.



Representation
Data is transferred in representations like JSON or XML.

REST API

Przykład REST w praktyce, REST = zasoby + metody HTTP + reprezentacje

- **Zasób:** obiekt, który można pobrać lub modyfikować (np. użytkownik, zdjęcie, wiadomość).
- **URI:** adres zasobu (np. `/api/users/`)
- **Reprezentacja:** dane opisujące zasób najczęściej w formacie **JSON** lub **XML**.
- **Operacje:** standardowe metody HTTP:
 - **GET** `/users` - Pobierz listę wszystkich użytkowników
 - **GET** `/users/123` - Pobierz dane użytkownika o ID = 123

<https://localhost/lecture14/api/version1.php/>

<https://localhost/lecture14/api/version1.php/1>

<https://localhost/lecture14/api/version1.php/123>

REST API

Przykład HTTP w standardzie REST API

- **Metody HTTP** to sposób **określenia intencji klienta względem zasobu na serwerze**.
- Każda metoda ma swoją specyficzną semantykę, która mówi, co klient chce zrobić z danym zasobem.
- W **REST API** najczęściej wykorzystuje się zestaw standardowych metod HTTP, które są zgodne z zasadami REST.
- Dzięki temu API staje się intuicyjne, spójne i zgodne z protokołem sieciowym.

REST API

Podstawowe metody HTTP w REST

- **GET** - Pobierz reprezentację zasobu. - Nie (niebezpieczna i idempotentna) - Pobranie listy użytkowników `/users` lub pojedynczego użytkownika `/users/123`
- **POST** - Utwórz nowy zasób lub wykonaj operację. - Tak (nie idempotentna) - Dodanie nowego użytkownika `/users` z danymi w ciele żądania
- **PUT** - Zastąp cały zasób nową reprezentacją. - Tak (idempotentna) - Zaktualizuj użytkownika `/users/123` całym obiektem
- **PATCH** - Częściowa modyfikacja zasobu. - Tak (idempotentna) - Zmiana pojedynczego pola użytkownika `/users/123`
- **DELETE** - Usuń zasób. - Tak (idempotentna) - Usunięcie użytkownika `/users/123`

<https://localhost/lecture14/api/version2.php>

REST API

Testowanie przykładu

- Do testowania działania implementacji REST API, a szczególności metod HTTP takich jak **PUT**, **PATCH** oraz **DELETE**.
- Potrzebne będą programy które utworzą nam odpowiedzi request z niestandardową metodą HTTP.
- Jedną z najpopularniejszych aplikacji z graficznym interfejsem do testowania endpointów jest **Postman** oraz **Bruno**.
- Czasami stworzenie odpowiedniego zapytania będzie tylko możliwe w **CLI**.
- Idealnym do tego celu narzędziem jest **CURL**.

REST API

CURL przykłady użycia

- **Pobranie listy użytkowników:**

```
curl http://localhost/lecture14/api/version2.php
```

- **Dodanie użytkownika:**

```
curl -X POST -H "Content-Type: application/json" -d '{"name": "Anna Nowak",  
"email": "anna@example.com"}' http://localhost/lecture14/api/version2.php
```

- **Aktualizacja użytkownika:**

```
curl -X PUT -H "Content-Type: application/json" -d '{"email":  
"nowyEmail@example.com"}' http://localhost/lecture14/api/version2.php/4
```

- **Usunięcie użytkownika:**

```
curl -X DELETE http://localhost/lecture14/api/version2.php/4
```


REST API

Jak statusy HTTP wspierają REST ?

- REST opiera się na wykorzystaniu standardowych statusów HTTP, które przekazują klientowi:
 - Czy operacja zakończyła się sukcesem?
 - Jeśli nie – dlaczego?
 - Co klient powinien zrobić dalej?
- Dzięki temu klient może reagować odpowiednio na odpowiedź, np. wyświetlić komunikat o błędzie, zapytać o ponowne uwierzytelnienie, czy przekierować użytkownika.
- W standardzie REST przy błędach zwraca się także opis błędu w formacie JSON, aby klient mógł łatwo zinterpretować problem.

REST API

Przykłady statusy HTTP w kontekście REST

- **200 OK** - Najczęściej używany status oznaczający, że żądanie zostało poprawnie zrealizowane.
- **201 Created** - Oznacza, że zasób został pomyślnie utworzony (zazwyczaj po POST).
- **204 No Content** - Oznacza, że żądanie zostało wykonane, ale nie ma żadnej treści do zwrócenia. (DELETE lub PUT)
- **400 Bad Request** - Żądanie jest niepoprawne lub niekompletne (np. błędny JSON, brak wymaganych pól, błąd walidacji).
- **401 Unauthorized** - Klient nie jest uwierzytelniony.
- **403 Forbidden** - Klient jest uwierzytelniony, ale nie ma uprawnień do wykonania operacji.

REST API

Przykłady statusy HTTP w kontekście REST cd.

- **404 Not Found** - Żądany zasób nie istnieje (np. /users/999 nie znaleziono).
- **405 Method Not Allowed** - Metoda HTTP nie jest dozwolona dla danego zasobu.
- **409 Conflict** - Konflikt w stanie zasobu, np. próba stworzenia zasobu, który już istnieje.
- **500 Internal Server Error** - Ogólny błąd serwera, oznacza, że coś poszło nie tak po stronie serwera.

REST API

Zmiana statusów HTTP przy pomocy PHP

- W PHP możesz zmienić kod statusu HTTP za pomocą funkcji `header()`.
- Lepszą funkcją jest `http_response_code()`, która umożliwia zarówno ustawienie, jak i odczytanie bieżącego kodu odpowiedzi HTTP.
- Aby zmienić kod statusu, wystarczy wywołać tę funkcję z odpowiednim parametrem, np. `http_response_code(404)`, co spowoduje ustawienie kodu odpowiedzi na 404.

<https://localhost/lecture14/api/version3.php>

① REST API

② Base64

③ JWT

Base64

Czym jest Base64?

- **Base64** to sposób kodowania danych binarnych (np. plików, obrazów, dźwięków, dokumentów) do postaci tekstowej, która może być bezpiecznie przesyłana lub przechowywana w systemach działających na znakach tekstowych.
- Jest to schemat kodowania, który konwertuje dowolne dane binarne na ciąg znaków z ograniczonego zbioru 64 znaków ASCII.

Base64

Dlaczego powstał Base64?

- W wielu systemach komputerowych (np. protokoły sieciowe, e-maile, REST API, XML, JSON) nie można przysyłać lub przechowywać surowych danych binarnych ponieważ:
 - Mogą zawierać nieczytelne lub specjalne znaki, które mają znaczenie w danym protokole (np. znaki sterujące, zero byte).
 - REST API, gdy nie można użyć **multipart/form-data**, albo w integracjach z systemami wymagającymi **JSON**.
 - Systemy mogą oczekiwać danych wyłącznie w formie tekstowej (ASCII/UTF-8).
 - Dane binarne mogłyby się uszkodzić lub być źle interpretowane.
 - Base64 powstał, aby zamienić dane binarne na bezpieczny, tekstowy format bez utraty informacji.

Base64

Zastosowania Base64

- **E-mail (MIME)** - kodowanie załączników, bo e-mail historycznie przesyła tekst ASCII.
- **Przesyłanie plików w JSON** - gdzie dane muszą być tekstowe.
- **Autoryzacja HTTP Basic Auth** - login i hasło są kodowane w Base64.
- **Przechowywanie obrazów w CSS lub HTML** - jako data URI (`data:image/png;base64,...`).
- **Protokół XML** - gdzie dane binarne trzeba wstawić jako tekst.

Base64

Kodowanie Base64 w PHP

- W PHP kodowanie danych w formacie **Base64** odbywa się za pomocą funkcji:
 - **base64_encode()** – koduje dane do Base64
 - **base64_decode()** – dekoduje dane z Base64

<https://localhost/lecture14/base64/textExample.php>

<https://localhost/lecture14/base64/imageExample.php>

<https://localhost/lecture14/base64/htmlExample.php>

<https://localhost/lecture14/base64/uploadExample.php>

<https://localhost/lecture14/base64/downloadExample.html>

① REST API

② Base64

③ JWT

Co to jest JWT?

- **JWT (JSON Web Token)** to otwarty standard (RFC 7519) definiujący sposób bezpiecznego przesyłania informacji między stronami jako obiekt JSON.
- Informacje te są podpisane cyfrowo, co zapewnia ich integralność i możliwość weryfikacji autentyczności.
- Token JWT składa się z trzech części rozdzielonych kropkami:
xxxxxx.yyyyyy.zzzzzz

Struktura JWT

- **Header (nagłówek)** – zawiera informacje o typie tokena i algorytmie podpisu.
- **Payload (ładunek)** – zawiera dane (claims), np. identyfikator użytkownika, role, czas wygaśnięcia:
- **Signature (podpis)** – podpis HMAC lub RSA/ECDSA nad pierwszymi dwiema częściami, w celu zapewnienia integralności.

Przykład tokenu JWT:

<https://jwt.io/>

JWT

JWT w uwierzytelnianiu i autoryzacji

- W systemach opartych o REST, JWT często służy do uwierzytelniania użytkowników. Proces wygląda następująco:
 - ➊ Logowanie (Authentication): Klient (np. aplikacja SPA) wysyła dane logowania (login/hasło) do serwera autoryzacji (np. `/api/login`).
 - ➋ Wygenerowanie tokena: Po poprawnym uwierzytelnieniu serwer generuje JWT i odsyła go do klienta.
 - ➌ Przechowywanie tokena: Klient zapisuje token (najczęściej w **localStorage** lub **sessionStorage** lub w bezpiecznym **ciasteczku HttpOnly**).
 - ➍ Używanie tokena: Klient dołącza JWT do każdego żądania do API w nagłówku Authorization:
Authorization: Bearer <token>
 - ➎ Weryfikacja tokena: Backend weryfikuje podpis i datę ważności tokena przy każdym żądaniu.

Statelessness a JWT w REST

- W architekturze REST kluczową zasadą jest **bezstanowość** (ang. **statelessness**). Oznacza to, że serwer nie **przechowuje żadnych informacji o stanie klienta między żadaniami**.
- Dzięki JWT możliwe jest pełne stateless uwierzytelnianie, ponieważ:
 - Token zawiera wszystkie potrzebne dane (np. ID użytkownika, role, czas ważności).
 - Serwer nie musi utrzymywać sesji ani przechowywać informacji o zalogowanych użytkownikach.
 - Każde żądanie zawiera wszystkie potrzebne dane do autoryzacji właśnie w tokenie.

JWT

Implementacja JWT

- Aby w prosty sposób zaimplementować podstawowy mechanizm JWT można skorzystać z gotowych bibliotek udostępnionych na platformie **GitHub**.
- Jedną z nich jest **firebase/php-jwt**.
- Wystarczy przekopiować z katalogu [src/](#) dwa pliki ([JWT.php](#) oraz [Key.php](#)) do katalogu z naszym projektem.

Przykład:

<http://localhost/lecture14/jwt/>

Przykład logowania:

<http://localhost/lecture14/jwt/login.php>

Dziękuję za uwagę !

Bartosz Lauks